



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
scanf("%d", &n);

arr = (int*)malloc(n * sizeof(int));

if(arr == NULL)

printf("Memory not allocated.\n");

else

{

for (j = 0; j < n; ++j)

scanf("%d",&arr[j]);

}

heapSort(arr, n);

printf("Sorted array is: \n");

printArray(arr, n);

}
```

EXPERIMENT-4

AIM : Implement BFT and DFT for given graph, when graph is represented by a) Adjacency Matrix
b) Adjacency Lists

Program : BFT

a)Adjacency Matrix

```
#include<stdio.h>
#include <stdbool.h>
#define MAX 100
// BFS from given source s
void bfs(int adj[MAX][MAX], int V, int s) {

// Create a queue for BFS
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
int q[MAX], front = 0, rear = 0;

// Initially mark all the vertices as not visited
// When we push a vertex into the q, we mark it as
// visited
bool visited[MAX] = { false };

// Mark the source node as visited and enqueue it
visited[s] = true;
q[rear++] = s;

// Iterate over the queue
while (front < rear) {

    // Dequeue a vertex and print it
    int curr = q[front++];
    printf("%d ", curr);

    // Get all adjacent vertices of the dequeued vertex
    // If an adjacent has not been visited, mark it visited and enqueue it
    for (int i = 0; i < V; i++) {
        if (adj[curr][i] == 1 && !visited[i]) {
            visited[i] = true;
            q[rear++] = i;
        }
    }
}

// Function to add an edge to the graph
void addEdge(int adj[MAX][MAX], int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1; // Undirected graph
}

int main() {
    // Number of vertices in the graph
    int V = 5;

    // Adjacency matrix representation of the graph
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
int adj[MAX][MAX] = {0};
```

```
// Add edges to the graph
```

```
addEdge(adj, 0, 1);  
addEdge(adj, 0, 2);  
addEdge(adj, 1, 3);  
addEdge(adj, 1, 4);  
addEdge(adj, 2, 4);
```

```
// Perform BFS traversal starting from vertex 0  
printf("BFS starting from 0:\n");  
bfs(adj, V, 0);
```

```
return 0;
```

```
}
```

2(B): // Program to implement DFS

```
#include <stdio.h>
```

```
#define MAX_VERTICES 100
```

```
void DFS(int graph[MAX_VERTICES][MAX_VERTICES], int visited[MAX_VERTICES],  
int vertices, int start)
```

```
{
```

```
    printf("%d ", start);  
    visited[start] = 1;
```

```
    for (int i = 0; i < vertices; i++)
```

```
    {
```

```
        if (graph[start][i] == 1 && !visited[i])  
            DFS(graph, visited, vertices, i);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int vertices, edges;  
    printf("Enter the number of vertices: ");  
    scanf("%d", &vertices);  
    if (vertices <= 0 || vertices > MAX_VERTICES)
```

```
    {
```

```
        printf("Invalid number of vertices. Exiting...\n");
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
        return 1;
    }
// Initialize the adjacency matrix with zeros
int graph[MAX_VERTICES][MAX_VERTICES] = {0};
// Initialize the visited array with zeros
int visited[MAX_VERTICES] = {0};
printf("Enter the number of edges: ");
scanf("%d", &edges);
if (edges < 0 || edges > vertices * (vertices - 1))
{
    printf("Invalid number of edges. Exiting...\n");
    return 1;
}

// Input edges and construct the adjacency matrix
for (int i = 0; i < edges; i++)
{
    int start, end;
    printf("Enter edge %d (start end): ", i + 1);
    scanf("%d %d", &start, &end);

    // Validate input vertices
    if (start < 0 || start >= vertices || end < 0 || end >= vertices)
    {
        printf("Invalid vertices. Try again.\n");
        i--;
        continue;
    }

    graph[start][end] = 1;
    // For undirected graph, uncomment the following line:
    // graph[end][start] = 1;
}

// Input the starting vertex for DFS traversal
int startVertex;
printf("Enter the starting vertex for DFS traversal: ");
scanf("%d", &startVertex);

if (startVertex < 0 || startVertex >= vertices)
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
{  
    printf("Invalid starting vertex. Exiting...\n");  
    return 1;  
}  
  
printf("DFS Traversal Order: ");  
DFS(graph, visited, vertices, startVertex);  
return 0;  
}
```

EXPERIMENT-5

AIM:

Implement the string processing algorithms- Brute Force & Boyer Moore methods.

Program :

```
#include <stdio.h>  
char T[50],p[10];  
char C[26]={ 'a','b','c','d','e','f','g','h','i','k','l','m','n','o','p','q','r','s','t','u','v','x','y','z'};  
int L[26];  
  
void BuildLast(char p[], int m, int L[])  
{  
    int i,j;  
    char A;  
    for(i=0; i < 26; i++)  
        L[i] = -1; // initialize array  
    for (i = 0; i < m; i++)  
    {  
        A=p[i];  
        for(j=0;j<26;j++)  
            if (A==C[j]) break;  
        L[j]= i;  
    }  
}  
  
void main()  
{  
    int i,j,m,n,t,k,lo;  
    char A;  
    // clrscr();  
    printf("Enter the text without blank spaces:");  
    scanf("%s",T);
```